

LEARN

DRINK

JAVA™
TECHNOLOGY

LIVE PLAY



Developing Mission-Critical Applications using JDBC™ 2.0 Technology

Dr. Seth White

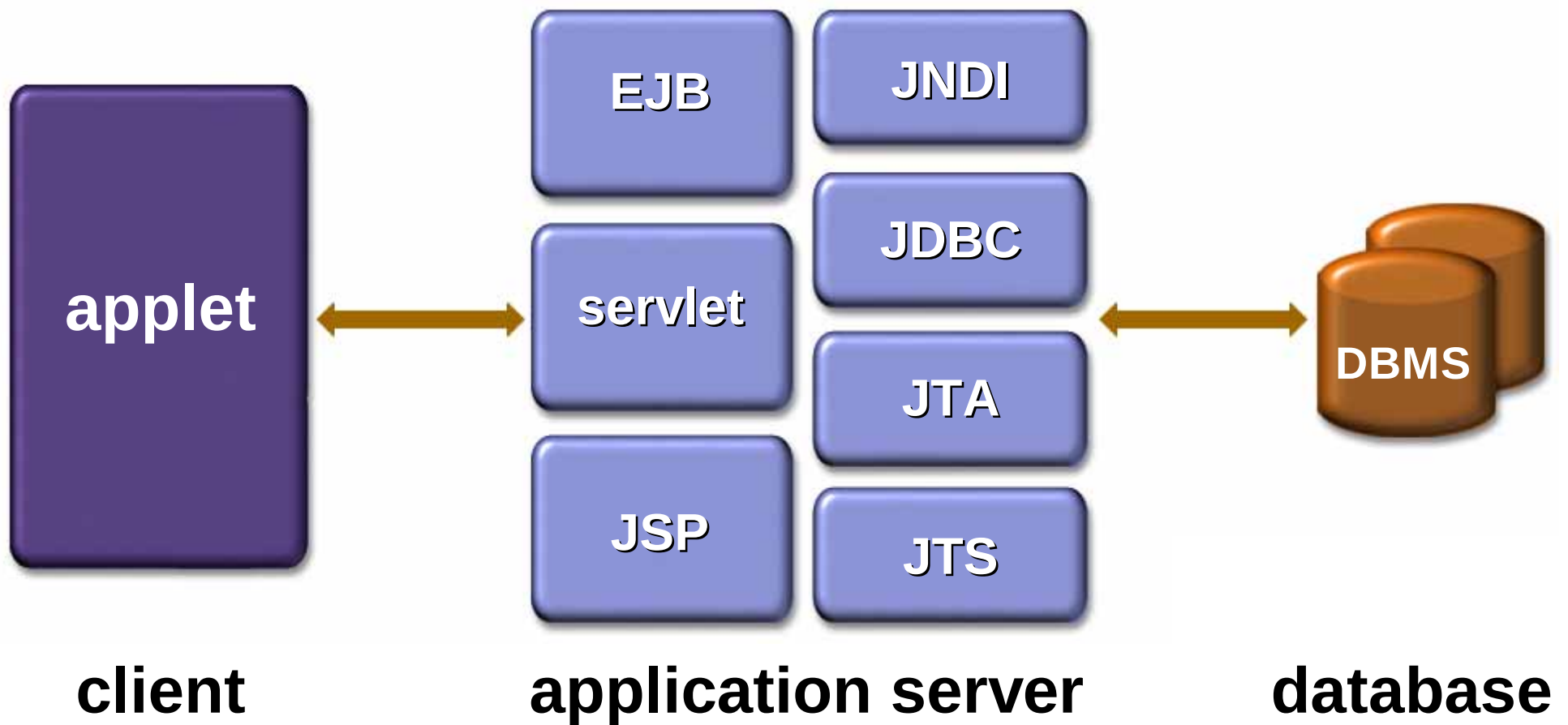
Member of Technical Staff, Sun Microsystems, Inc.

Agenda

- **Basic architecture**
- **Data sources**
- **Row sets**
- **Enterprise demo**



The Basic Architecture



JDBC Technology on the Server

- **Components**
- **Manageability**
- **Portability**
- **Performance**



JDBC and JNDI

- **Java Naming and Directory Interface™ (JNDI)**
- **Network-wide sharing of information**
 - Users
 - Machines
 - **Services**
 - Applications
- **Focus on connectivity**



Goals for JDBC and JNDI

- **Separate development and management**

```
Class.forName("SomeJDBCDriverClassName");  
Connection con = DriverManager.getConnection(  
    "jdbc:vendorX:machineY:portZ");
```

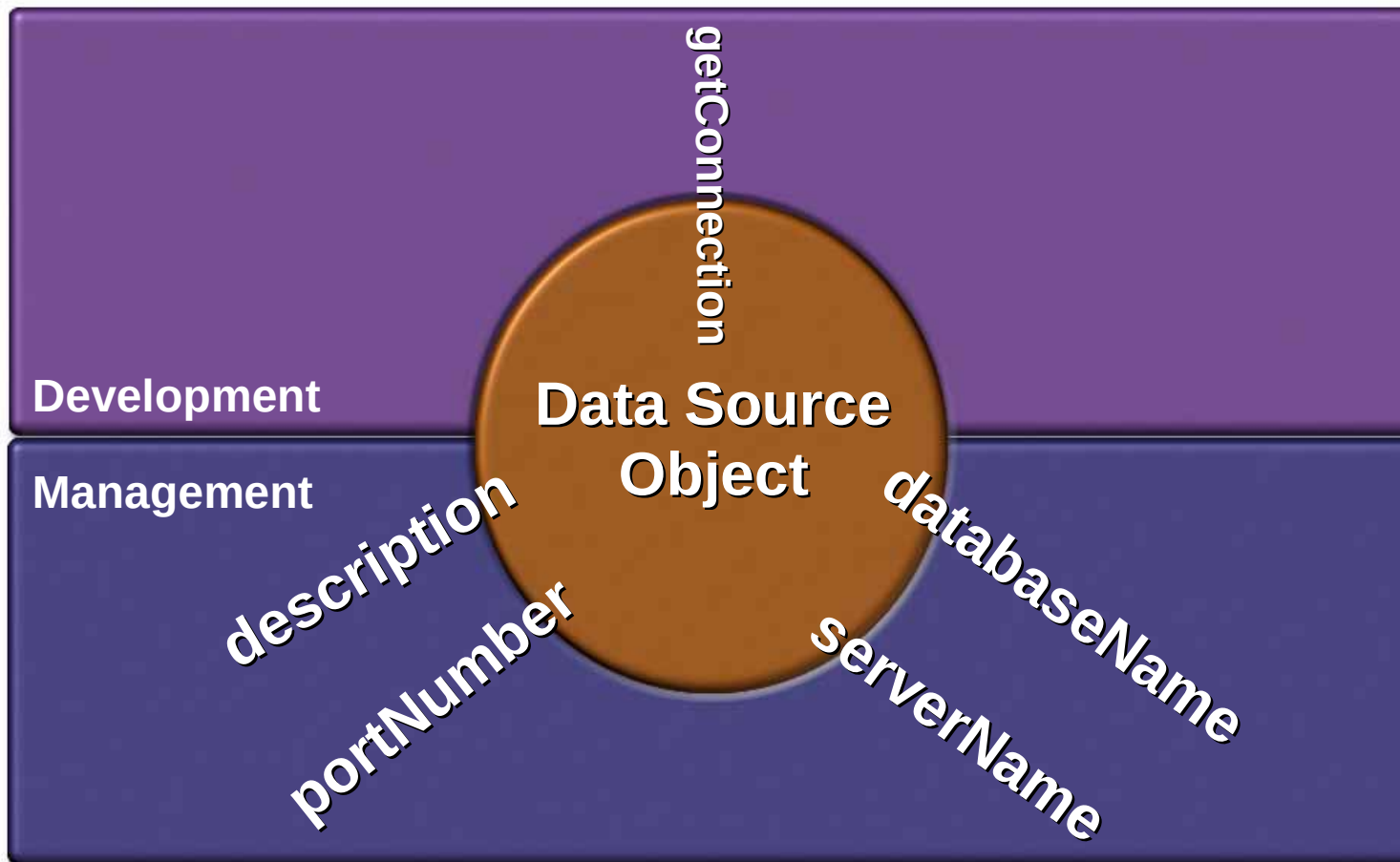
- **Provide additional services transparently**

- Connection pooling
- Distributed transactions

- **Portability for server applications**

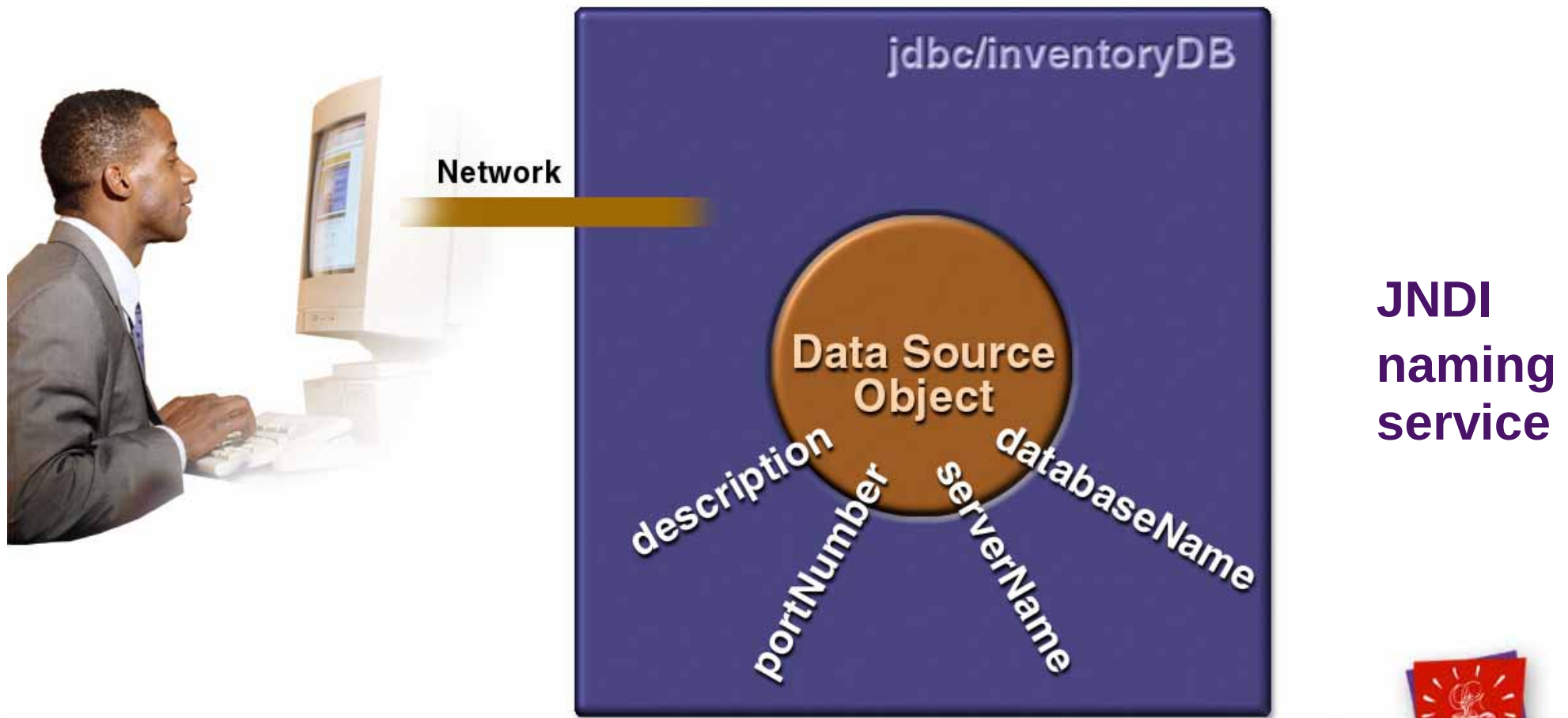


Data Source Objects

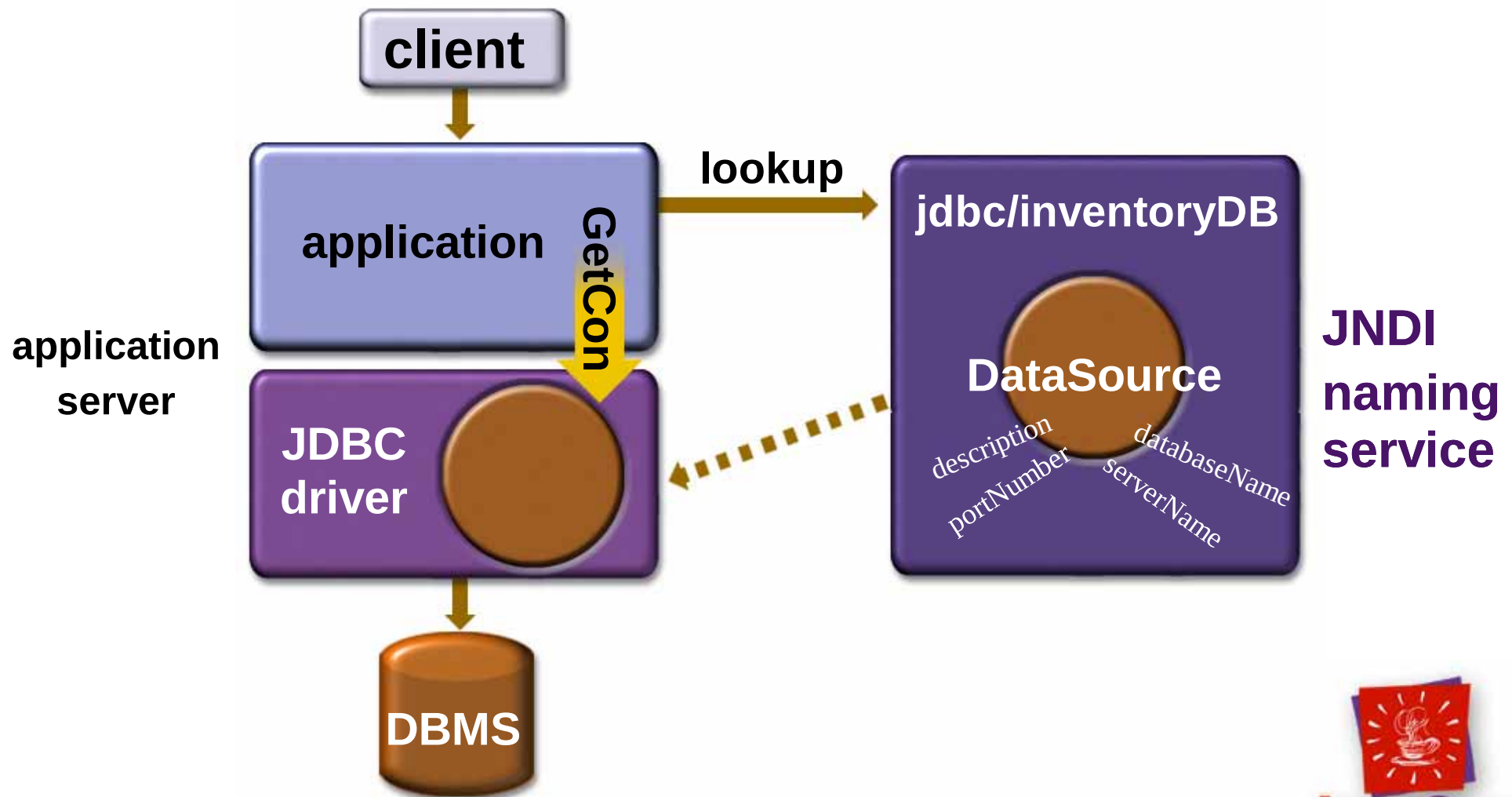


- Represents a real-world data source
- Create, initialize, and deploy using a tool

Deploying a Data Source



Connecting to a Data Source



Sample Component Code

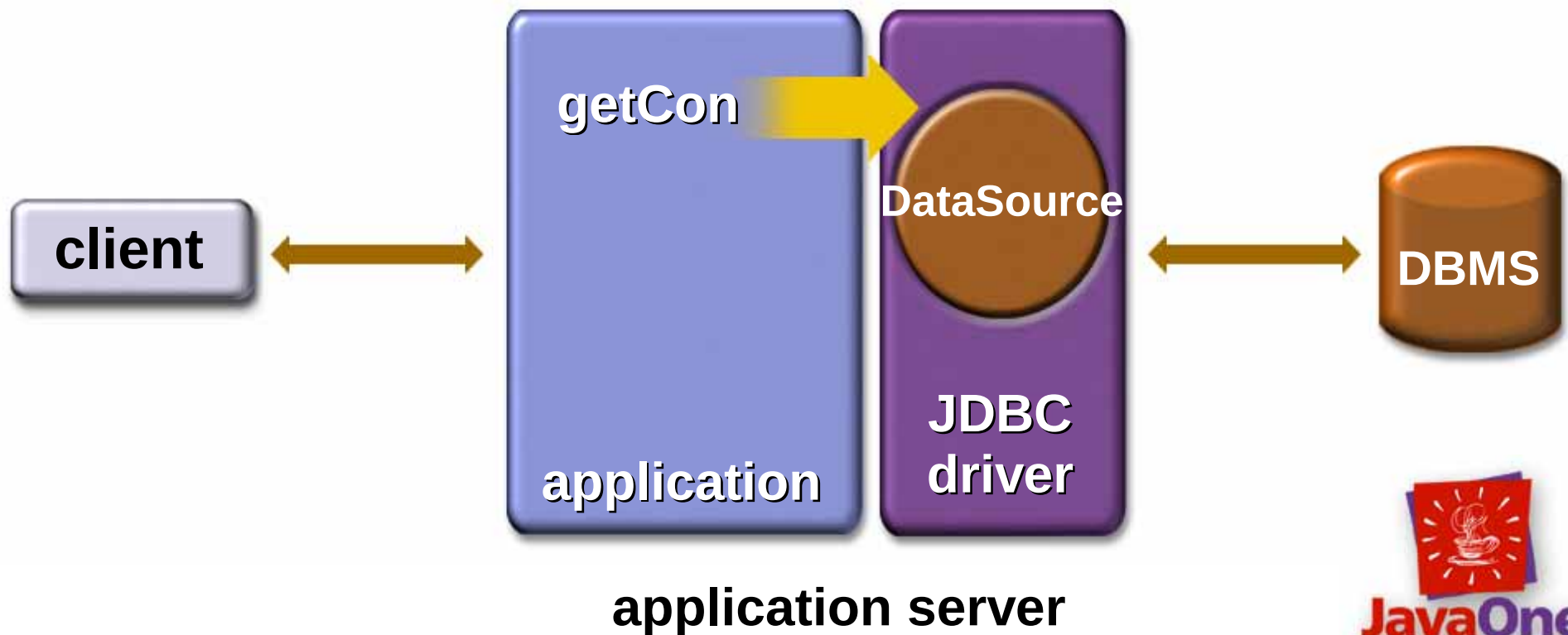
```
Connection con = null;

try {
    Context ctx = new InitialContext();
    DataSource ds = (DataSource)ctx.lookup(
        "jdbc/inventoryDB");
    con = ds.getConnection(user, password);
    .
    .
    .
} finally {
    if (con != null) con.close();
}
```

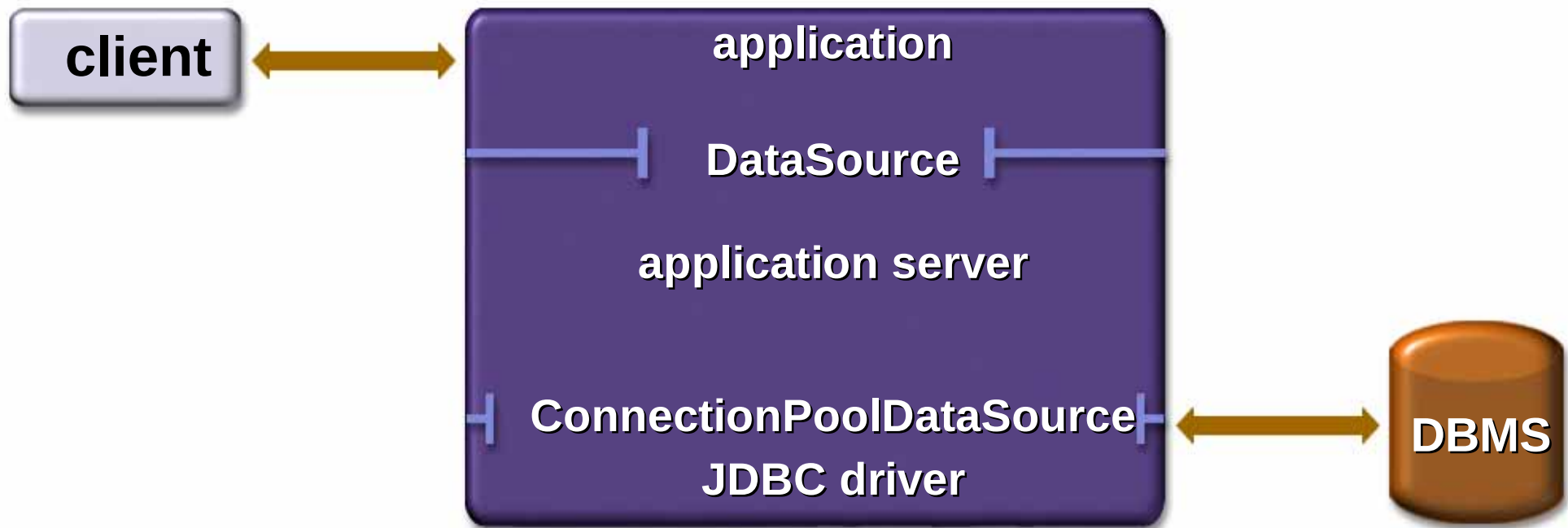


Drawbacks

- Slow performance
- No way to reuse connections

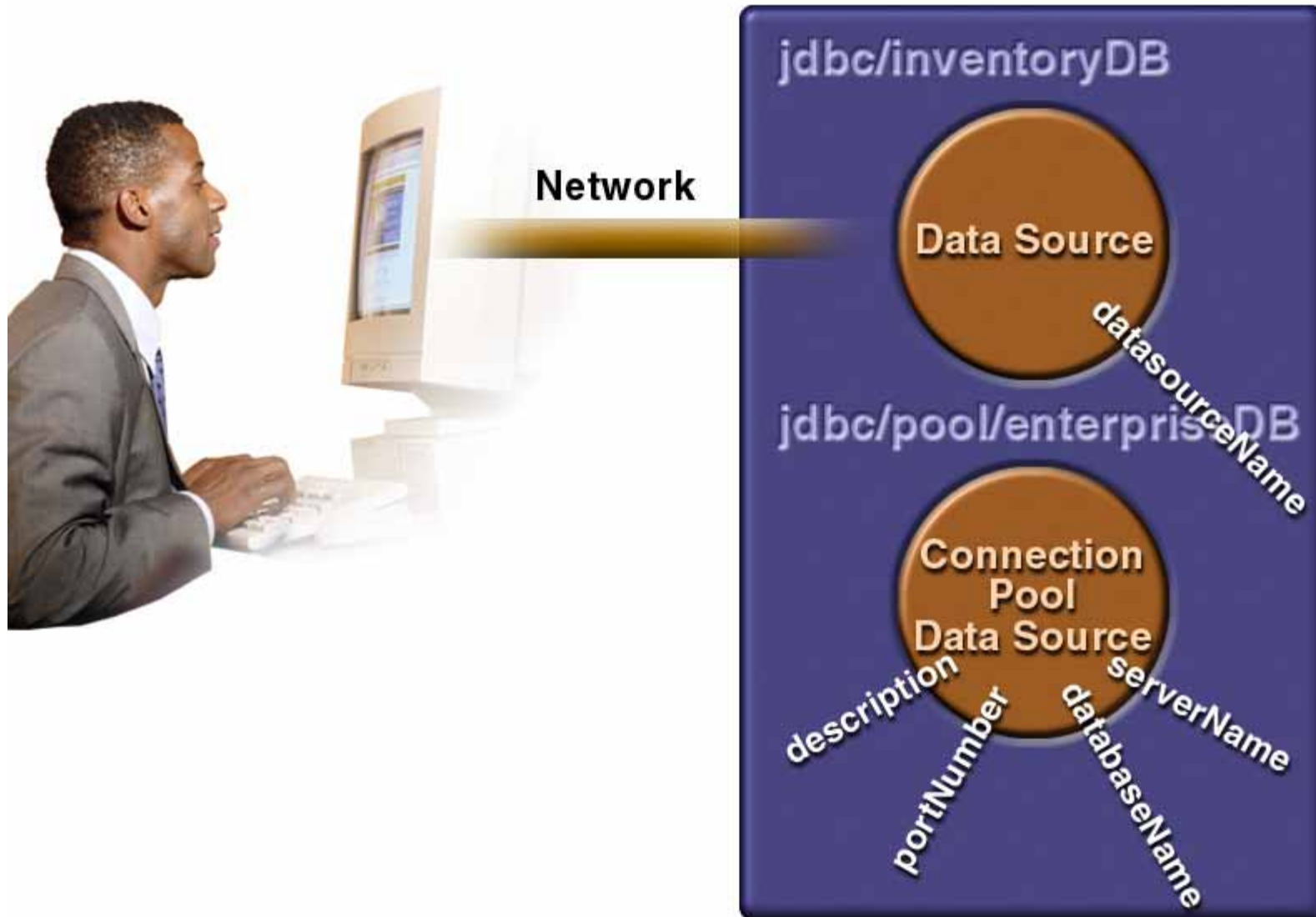


Connection Pooling



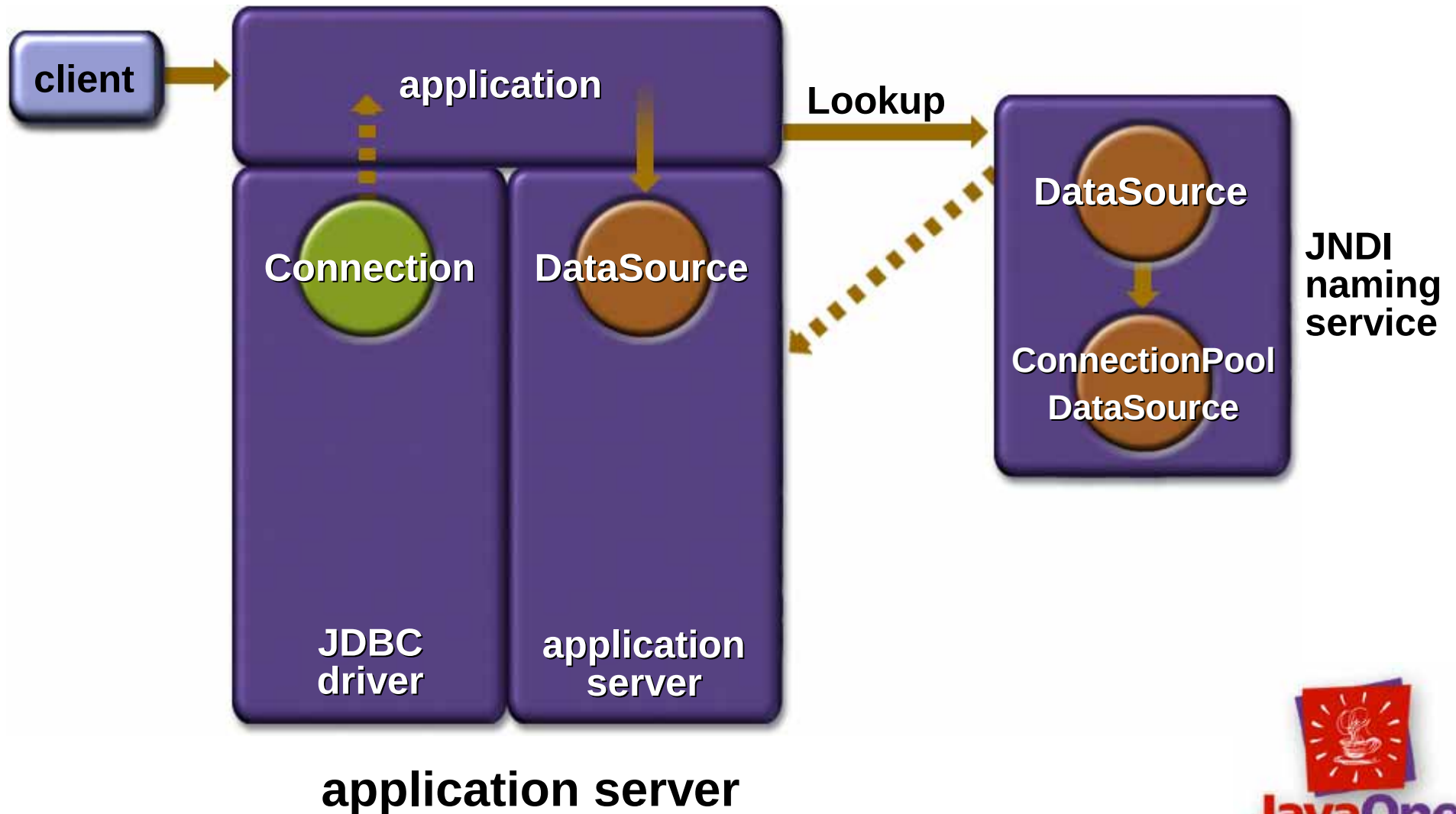
application server

Deployment with Connection Pooling

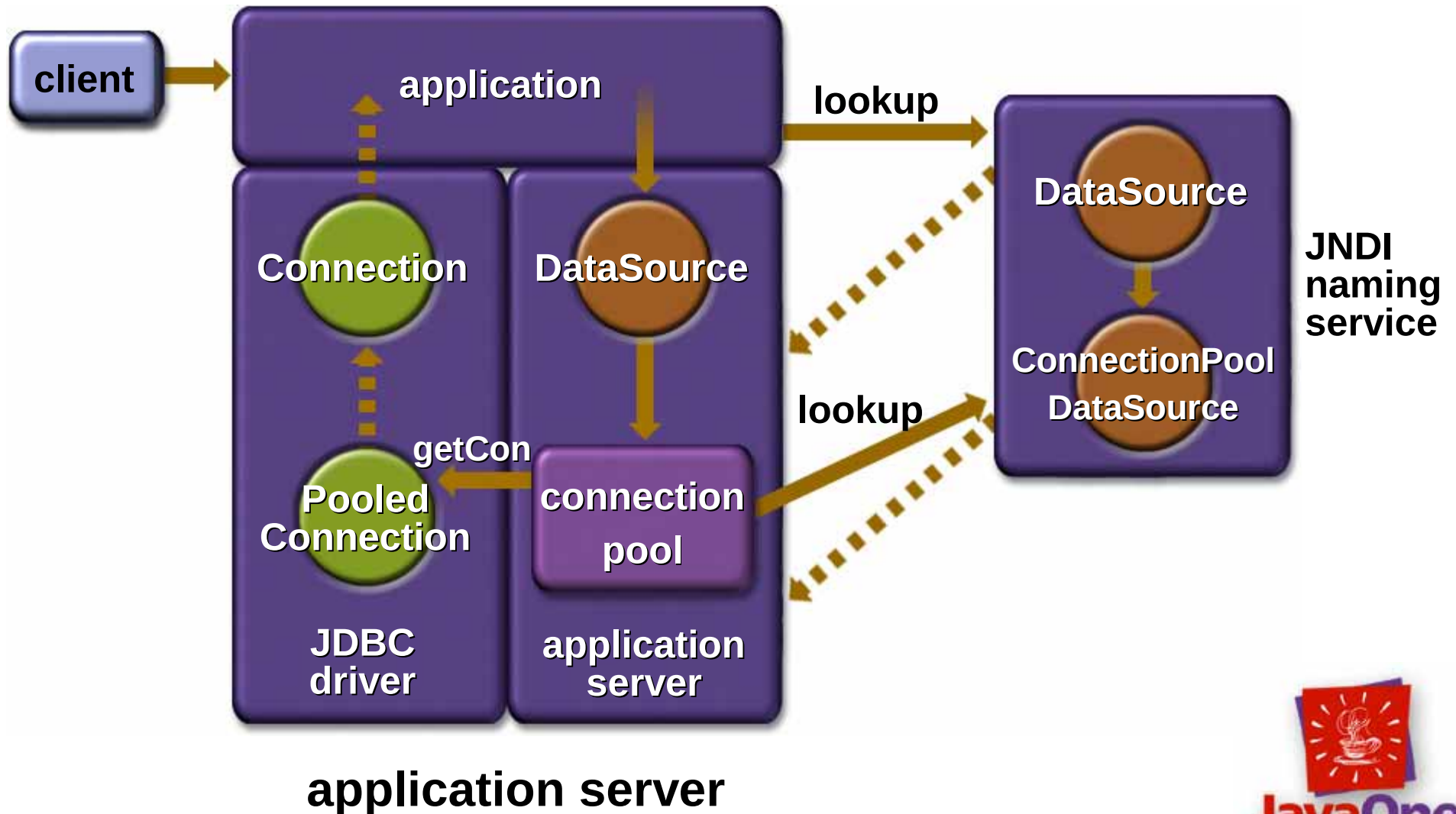


**JNDI
service
provider**

Connecting



Connecting (Detail)



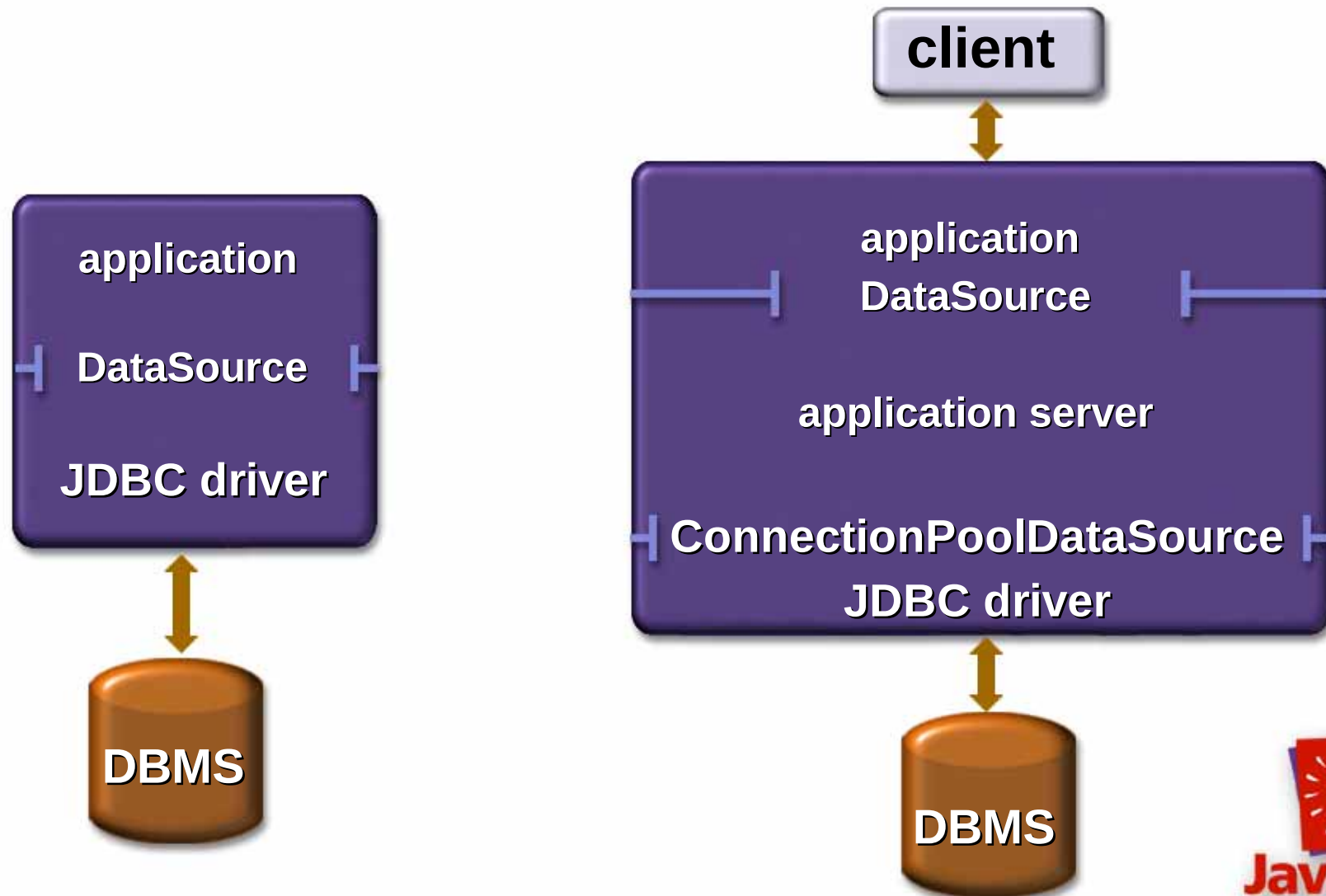
Sample Code (Hasn't Changed)

```
Connection con = null;

try {
    Context ctx = new InitialContext();
    DataSource ds = (DataSource)ctx.lookup(
        "jdbc/inventoryDB");
    con = ds.getConnection(user, password);
    .
    .
    .
} finally {
    if (con != null) con.close();
}
```



Plug and Play

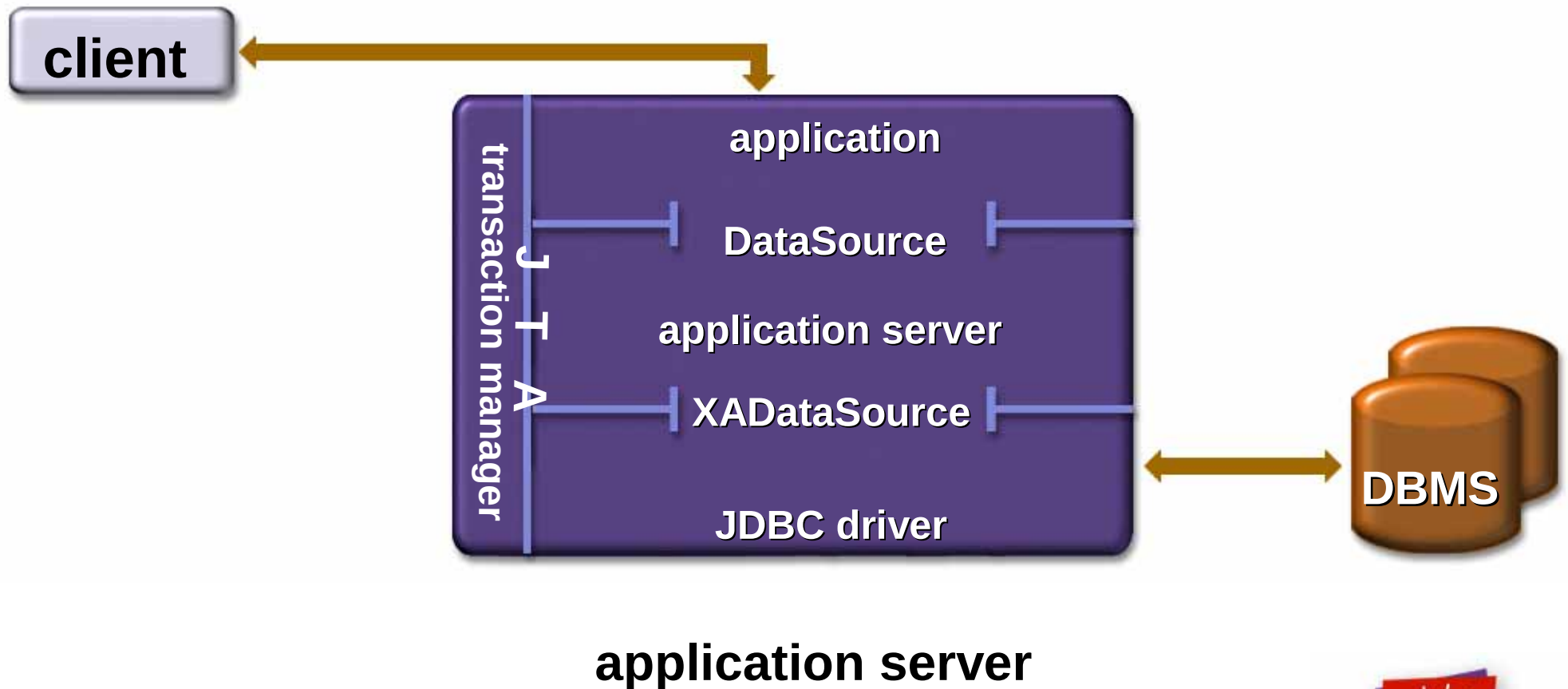


Limitations (Still)

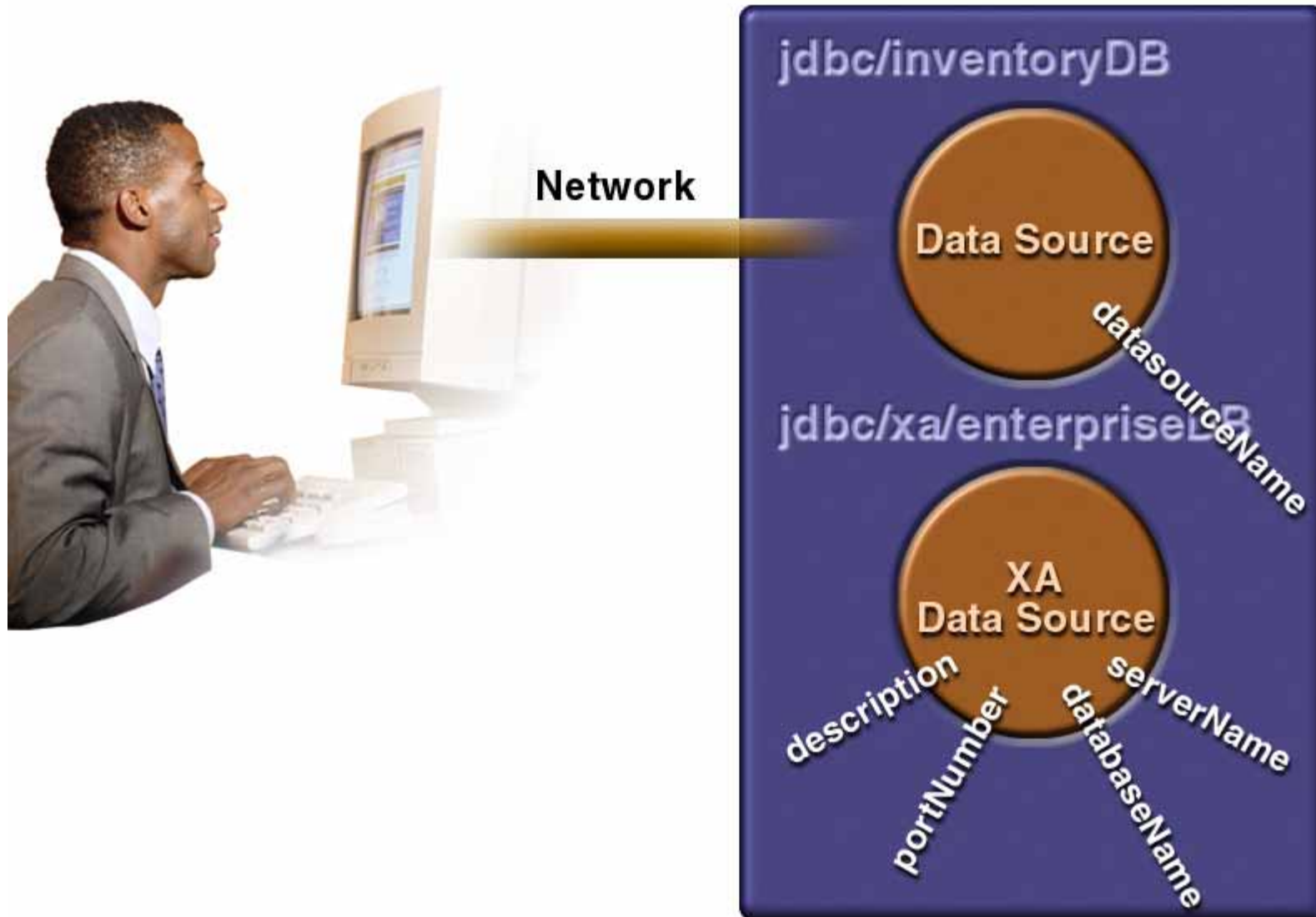
- Local transactions only
- Need a way to combine work
- Associate work on a connection with a distributed transaction



Distributed Transactions

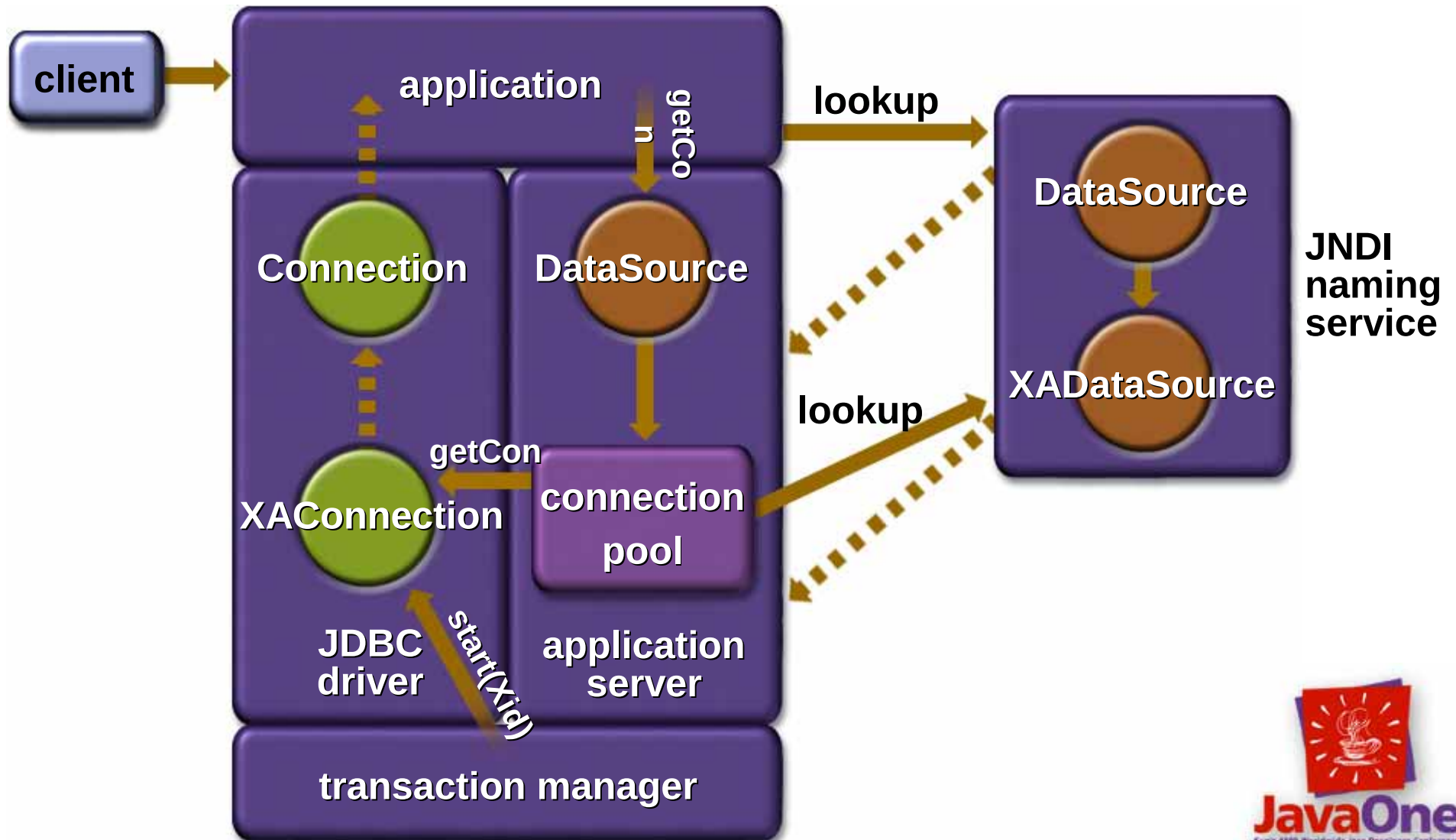


Deployment with Distributed Transactions



**JNDI
naming
service**

Connecting



Sample Code (Still the Same!)

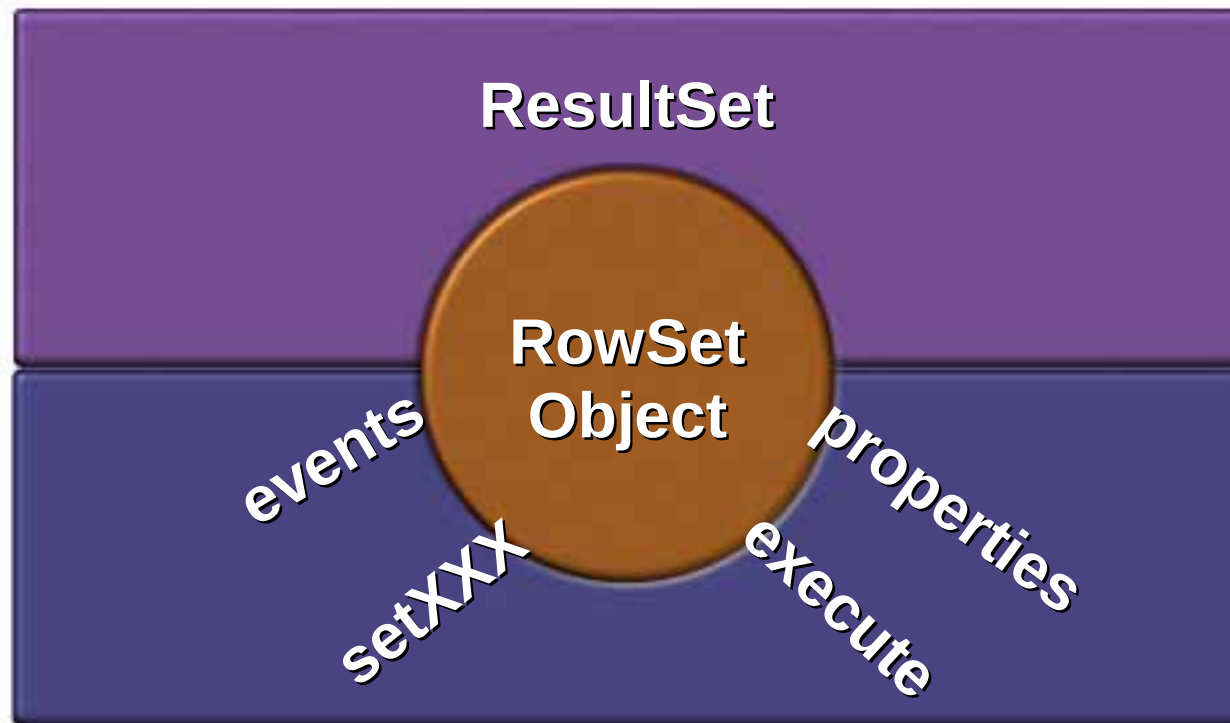
```
Connection con = null;

try {
    Context ctx = new InitialContext();
    DataSource ds = (DataSource)ctx.lookup(
        "jdbc/inventoryDB");
    con = ds.getConnection(user, password);
    .
    .
    .
} finally {
    if (con != null) con.close();
}
```



RowSets

- A JavaBeans™ technology component
- Can work with existing components



Advantages

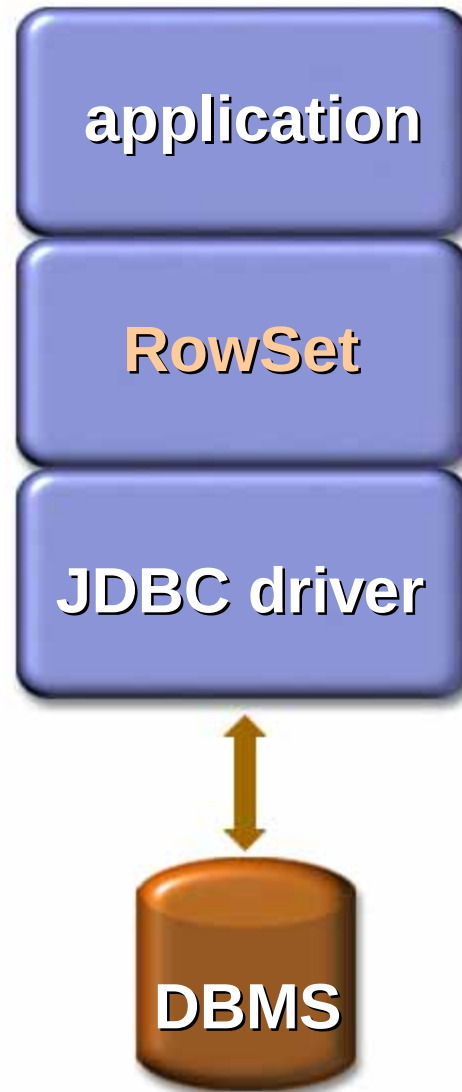
- **Provide additional services**
 - **Serializable**
 - **Scrolling**
 - **Caching**
 - **Disconnected access**
- **JavaBeans technology integration**

RowSet

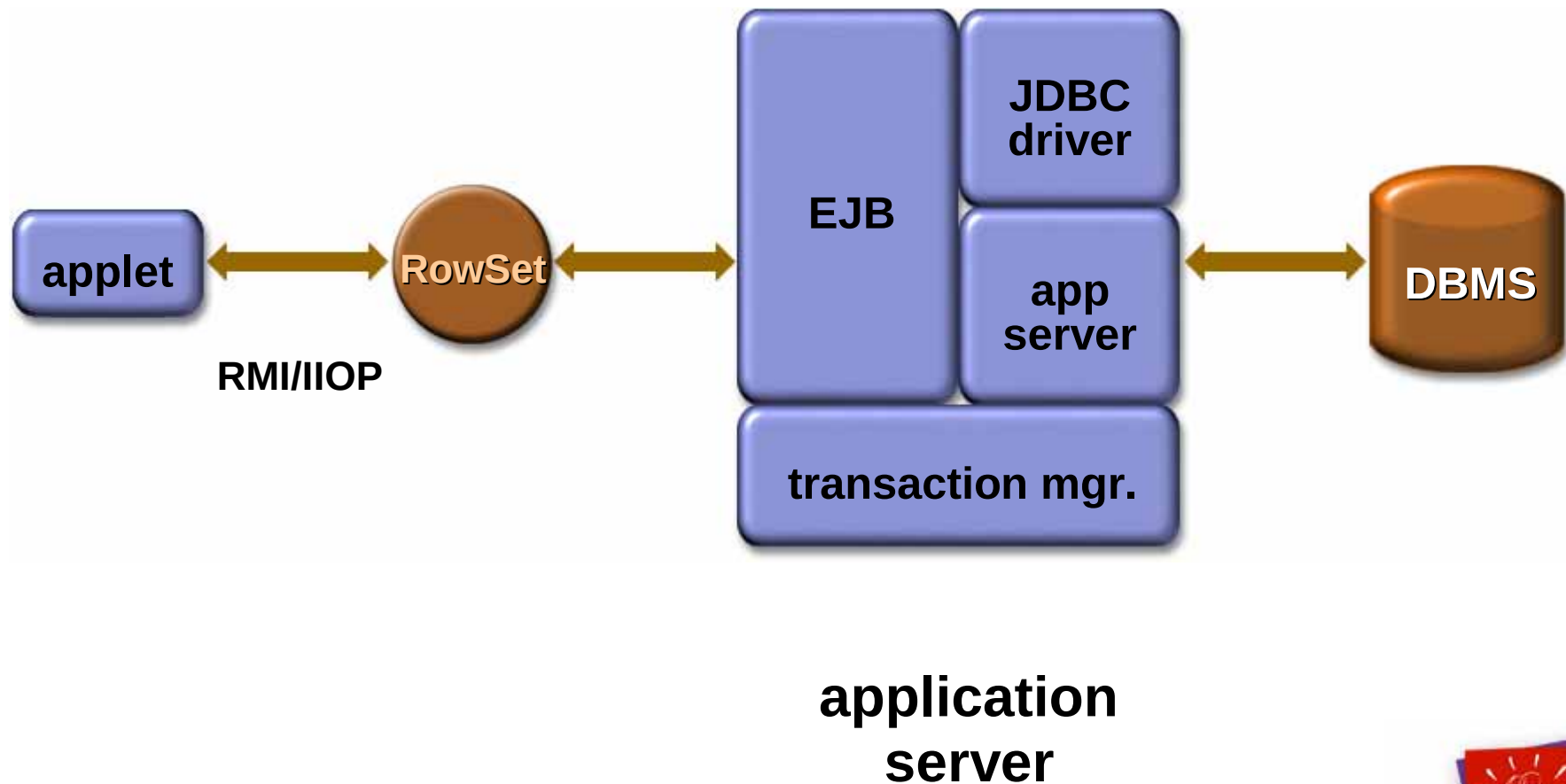
**JDBC
driver**



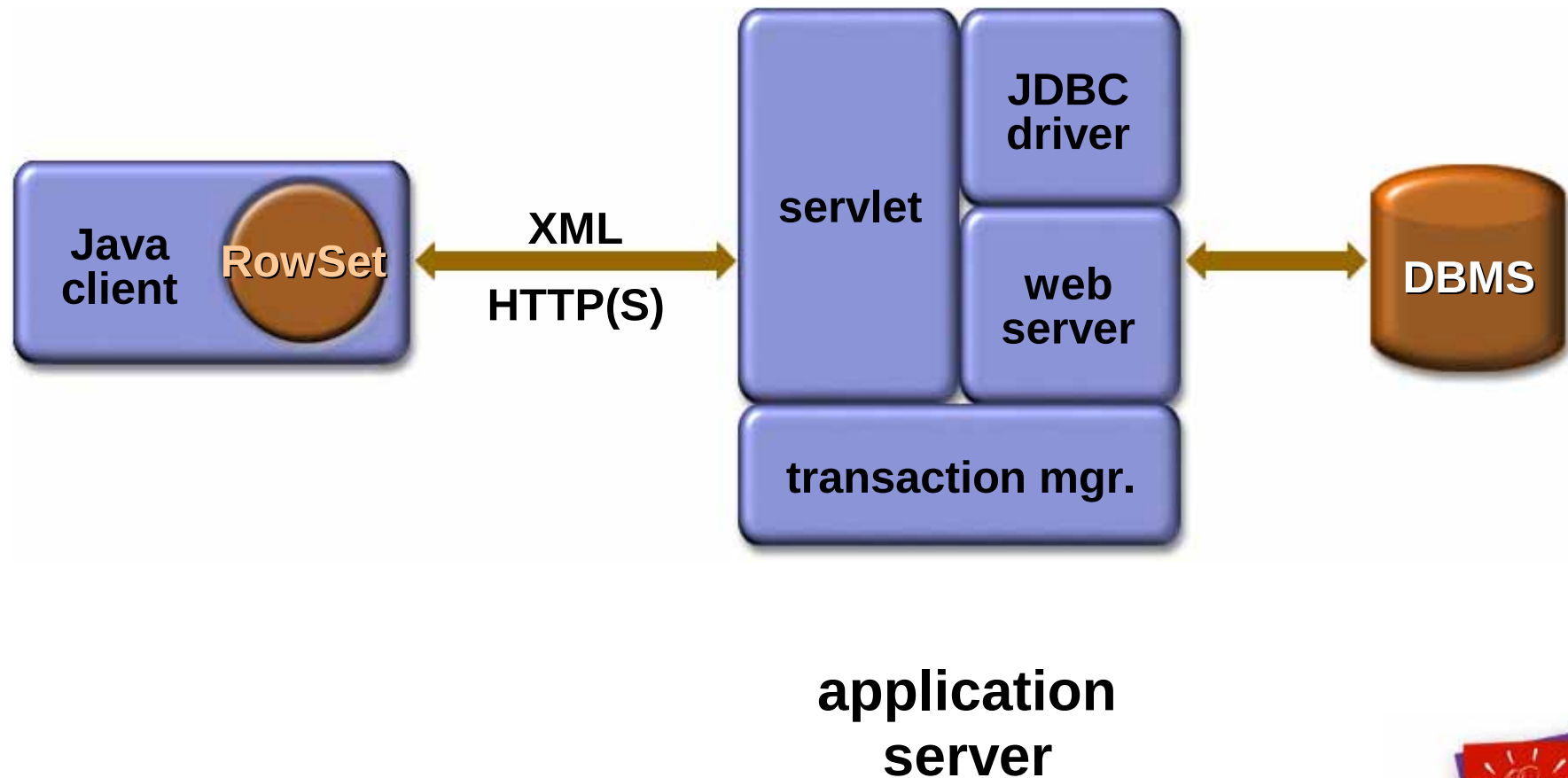
2-tier Architecture



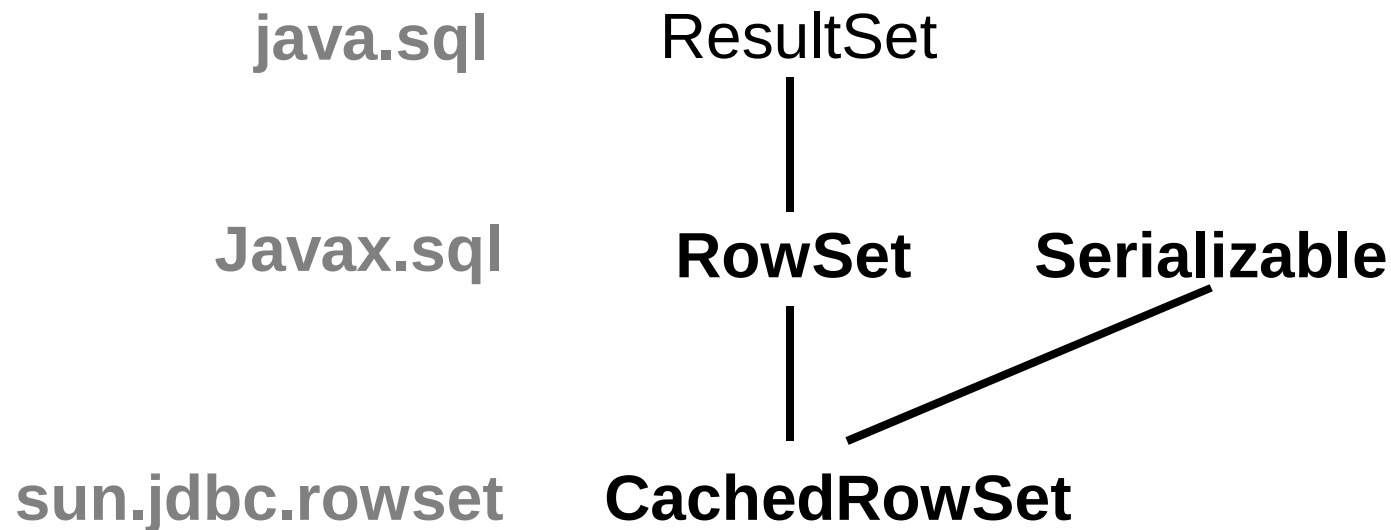
3-tier Architecture



3-tier Architecture (XML)



CachedRowSet



- **Breaking news: early access release**
 - <http://developer.java.sun.com/developer/earlyAccess>



Creating a CachedRowSet

```
import sun.jdbc.rowset;
```

```
CachedRowSet rset = new CachedRowSet();
```



Setting Properties

```
rset.setDataSourceName("jdbc/inventoryDB");
```

```
rset.setCommand(  
    "SELECT PART_NO, QUANTITY FROM PARTS");
```

```
rset.setType(ResultSet.TYPE_SCROLL_INSENSITIVE);
```

```
rset.setConcurrency(ResultSet.CONCUR_READ_ONLY);
```



Events

- **Types of events**
 - Cursor moved
 - Row changed
 - Rowset changed
- **Registering an event listener**

```
RowSetListener listener1 = new BarGraph();  
RowSetListener listener2 = new PieChart();  
rset.addListener(listener1);  
rset.addListener(listener2);
```



Enterprise JavaBeans™ (EJB) Technology Example

```
Class QueryBean implements SessionBean { ...
```

```
    RowSet doQuery(String query)  
                throws SQLException {  
        ...
```

```
        ResultSet rs = stmt.executeQuery(query);  
        CachedRowSet rset = new CachedRowSet();  
        rset.populate(rs);
```

```
        return rset;
```

```
    }
```

```
}
```



LEARN

DRINK

JAVA™
TECHNOLOGY

LIVE PLAY



JavaOne™
Sun's 1999 Worldwide Java Developer Conference*

EAT BREATHE

Demo

Technology Used

- EJB reference implementation from Sun
- JDBC™ CachedRowSet implementation
- MERANT SequeLink JDBC technology
- Oracle 8 XA compliant database (8.0.5)



How We Will Use the Technology

- **EJB reference implementation**
 - Coordinates the distributed transactions
 - Provides connection pooling framework
 - Holds the EJB
- **Cached Rowset**
 - Enhances data manipulations

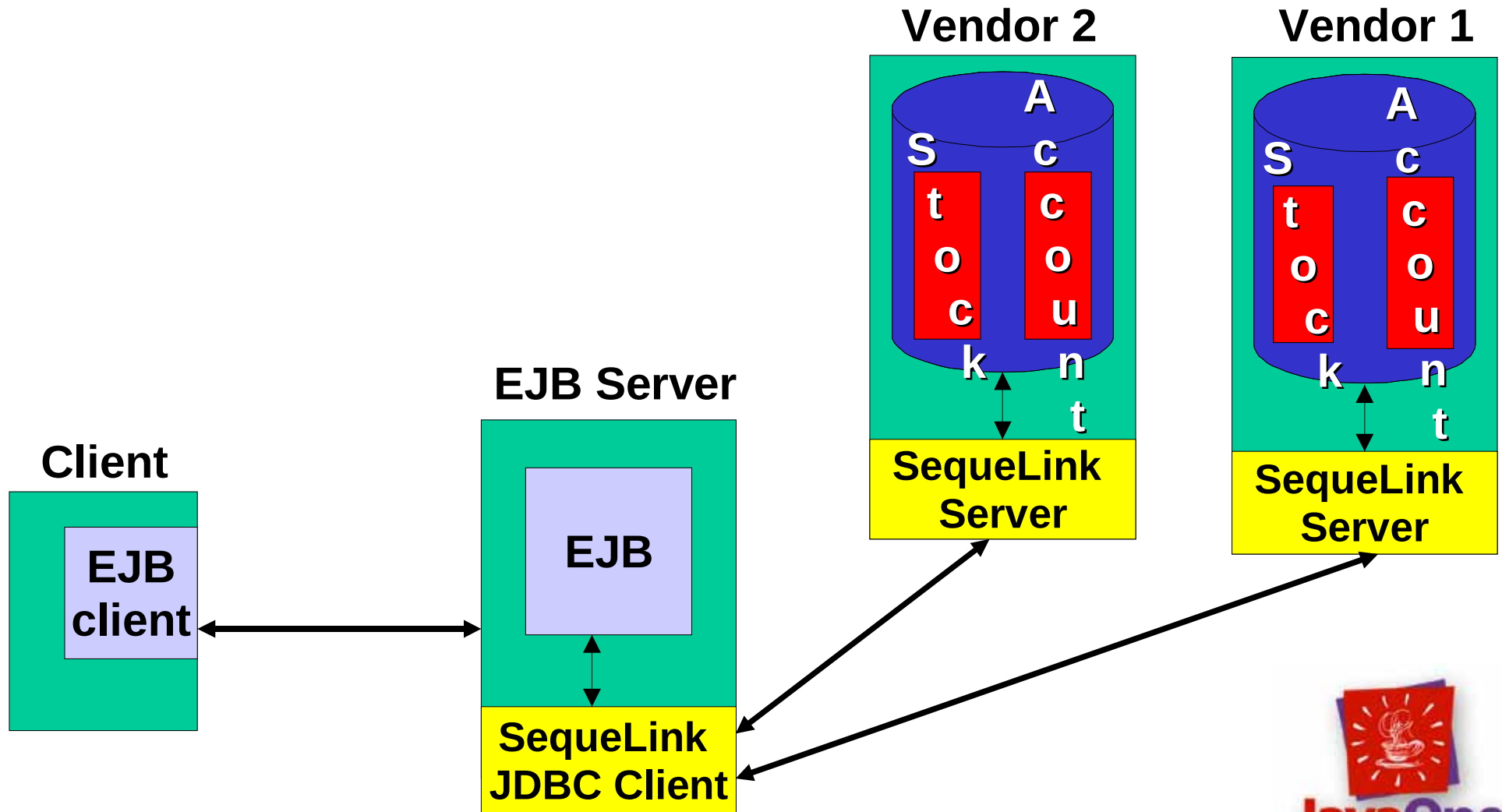


How We Will Use the Technology

- **MERANT SequeLink JDBC Technology**
 - Provides JDBC 2.0 access to the database
 - Enables connection pooling
 - Provides distributed transactions
- **Oracle 8:**
 - XA compliant Database Management System for both component vendors



Architecture



Business Methods Used

- **GetBestDeals**
- **GetStock**
- **GetOrders**
- **MakeOrder**



The GetBestDeals Method

- **Input: desired shipment days (1, 3 or 10)**
- **Retrieves entries from stock tables, in both databases, shippable in the desired number of days**
- **Combines both tables into one cached rowset**
- **If both tables contain the same product, the product with the lowest price is placed into the resulting cached rowset**
- **Resulting cached rowset is returned to the EJB client**



The GetStock Method

- Input: name of distributor
- Distributor's stock information is retrieved
- Resultset is placed into a CachedRowSet
- CachedRowSet is returned to the client



The GetOrders Method

- **Input: name of distributor**
- **Distributor's account information is retrieved**
- **Resultset is placed into a CachedRowSet**
- **CachedRowSet is returned to the client**



The MakeOrder Method

- **Input:**
 - Components that are ordered
 - Vendors that might be able to supply the components
 - Name and address of person that places the order



The MakeOrder Method (Cont.)

- **Connection to both databases is made for each component:**
 - If the ordered component is still available
 - Stock is decremented in the correct database
 - Account table is updated with the customer's order
 - Commit the transaction!
 - If the ordered component is no longer available then rollback the transaction



MakeOrder Code Example

```
// initialization of session
```

```
ut = context.getUserTransaction();  
ut.begin();  
con1 = getDBConnection(url1, user1, password1);  
con2 = getDBConnection(url2, user2, password2);  
stmt1 = con1.createStatement();  
stmt2 = con2.createStatement();
```



Code Example (Cont.)

```
Flag = true
for each <component> {
    if (distributer 1) stmt = stmt1;
    if (distributer 2) stmt = stmt2;
    rs = stmt.executeQuery(
        "select stock, arttype, price " +
        "from stock " +
        "where artnm like <componentname>");
    if (rs.getInt(stock) != 0)
        stmt.execute("update stock set stock=" +
            (rs.getInt("stock")-1) +
            " where artnm= <component>");
```



Code Sample (Cont.)

```
stmt.execute("insert into account values('" +  
            name + "', '" + address + "', " +  
            rs.getInt("arttype") + ", " +  
            rs.getInt("price") + ")");  
else  
    flag = false  
}  
if (flag)  
    ut.commit();  
else  
    ut.rollback();
```



Additional Information

- **Specifications**
 - <http://java.sun.com/products/jdbc>
- **Book**
 - **JDBC™ API Tutorial and Reference, Second Edition**
 - Available now
 - Addison-Wesley
 - ISBN 0-201-43328-1





JavaOneSM

Sun's 1999 Worldwide Java Developer Conference™